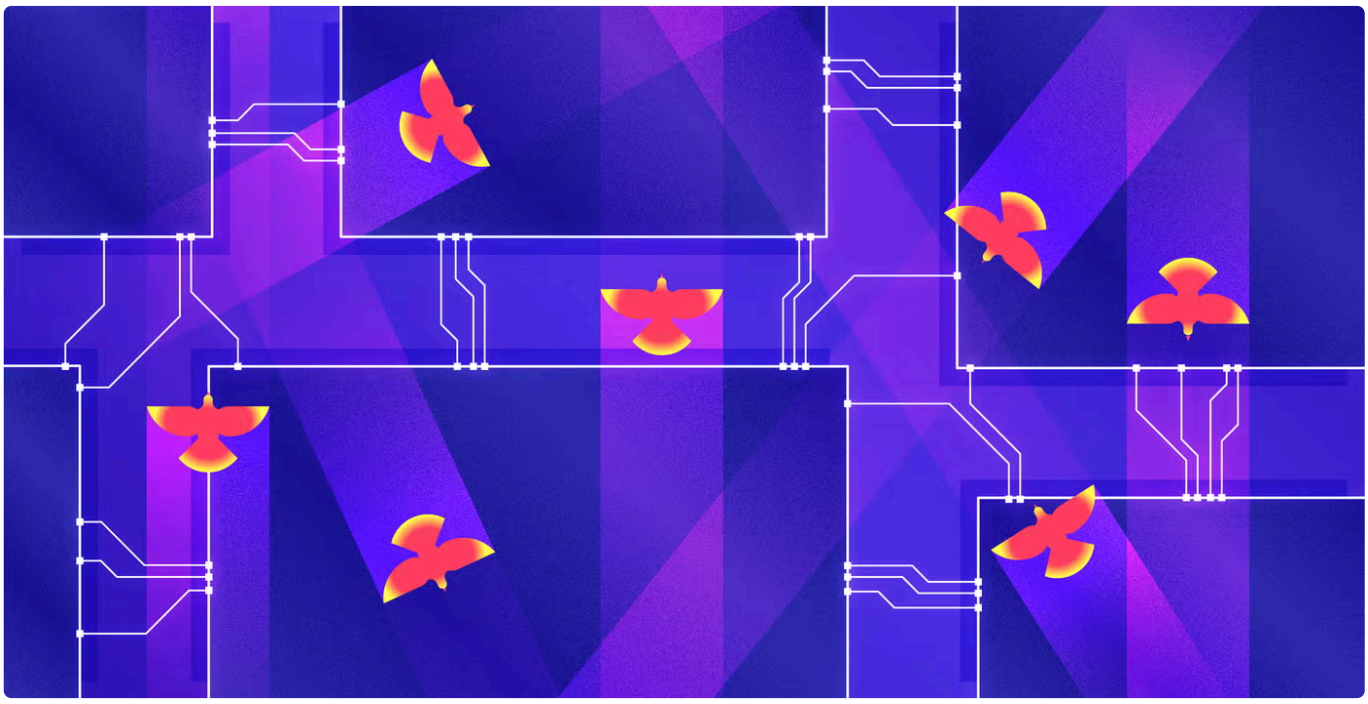
DASH NYC, June 9-10 | AI + Observability

DATADOG

ActiveMQ architecture and key metrics

INFRASTRUCTURE MONITORING APACHE ARCHITECTURE ACTIVEMQ MESSAGE QUEUE

PUBLISHED	**Dec 4, 2018**
UPDATED	**Sep 13, 2021**
READ TIME	**27m**





David M. Lentz

Editor's note (September 2021): This post has been updated to include new information about ActiveMQ Artemis.

Apache ActiveMQ is message-oriented middleware (MOM), a category of software that sends messages between applications. Using standards-based, asynchronous communication, ActiveMQ allows loose coupling of the services that make up an application, which is often foundational to enterprise messaging and distributed applications.

ActiveMQ is a Java-based open source project developed by the Apache Software Foundation. Apache currently offers two versions of ActiveMQ: Classic and Artemis. Once Artemis evolves to include all of the features available in the Classic version, Apache will support only a single version.

DASH NYC, June 9-10 | AI + Observability

DATADOG

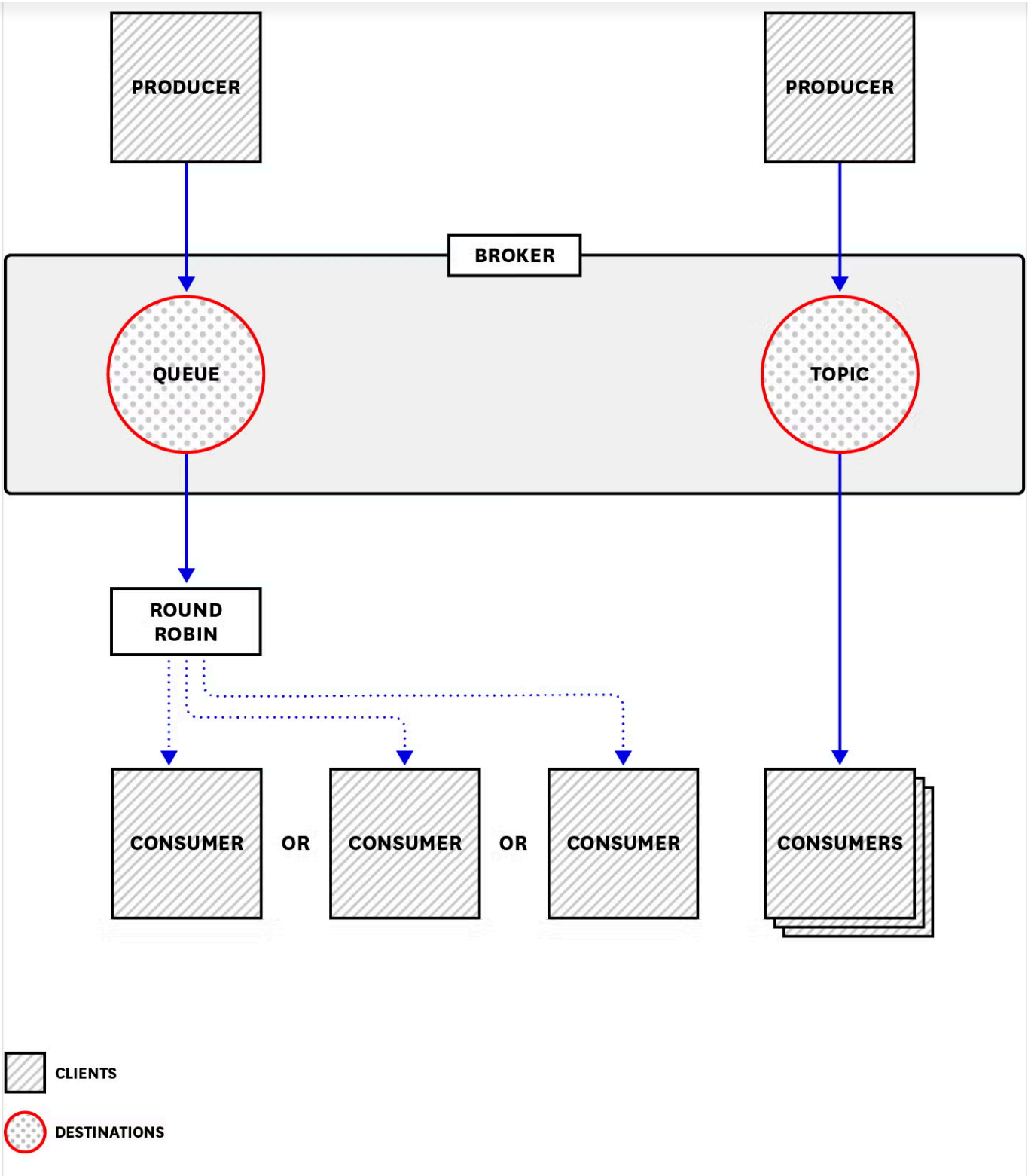
In this post, we'll look at how ActiveMQ works and explore some key ActiveMQ metrics you can monitor to understand the performance of your messaging infrastructure.

How does ActiveMQ work?

ActiveMQ sends messages between client applications—**producers**, which create messages and submit them for delivery, and **consumers**, which receive and process messages. The ActiveMQ **broker** routes each message through a messaging endpoint called a **destination** (in ActiveMQ Classic) or an **address** (in Artemis). Both ActiveMQ versions are capable of **point-to-point** messaging—in which the broker routes each message to one of the available consumers in a round-robin pattern—and **publish/subscribe** (or "pub/sub") messaging—in which the broker delivers each message to every consumer that is subscribed to the topic (in ActiveMQ Classic) or address (in ActiveMQ Artemis). The diagram below illustrates the components in an ActiveMQ Classic deployment.

DASH NYC, June 9-10 | AI + Observability

DATADOG



The ActiveMQ Classic architecture contains the broker, destinations, and client applications.

As shown above, Classic sends point-to-point messages via queues and pub/sub messages via topics. Artemis, on the other hand, uses queues to support both types of messaging and uses routing types to impose the desired behavior. In the case of point-to-point messaging, the broker sends a message to an address configured with the `anycast` routing type, and the message is placed into a queue where it will be retrieved by a single consumer. (Any `anycast` address typically has a single queue, but it can contain multiple queues if

DASH NYC, June 9-10 | AI + Observability

ActiveMQ implements the functionality specified in the **Java Message Service (JMS)** API, which defines a standard for creating, sending, and receiving messages. ActiveMQ client applications—producers and consumers—written in Java can use the JMS API to send and receive messages. Additionally, both Classic and Artemis support non-JMS clients written in Node.js, Ruby, PHP, Python, and other languages, which can connect to the ActiveMQ broker via the AMQP, MQTT, and STOMP protocols.

ActiveMQ sends messages asynchronously, so consumers don't necessarily receive messages immediately. The producer's task of composing and sending a message is disconnected from the consumer's task of fetching it. Because ActiveMQ uses a broker as an intermediary, producers and consumers are independent (and even unaware) of each other. As soon as a producer sends a message to a broker, its task is complete, regardless of whether or when a consumer receives the message. Conversely, when a consumer receives a message from a broker, it does so without knowledge of the producer that created the message.

This type of arrangement, in which clients function without knowledge of one another, is known as loose coupling. The benefits of loose coupling include:

- High throughput: Because producers don't need to wait for acknowledgment from the consumer or broker, they can send messages quickly. ActiveMQ can achieve throughput of thousands of messages per second.
- Flexibility: Clients can be temporarily unavailable, dynamically added to the environment, and even rewritten in a new language without affecting other clients or causing errors in the messaging process.
- Heterogeneity: Clients operate independently, communicating with the ActiveMQ broker but not directly with one another. As a result, they may be written in any of the languages ActiveMQ supports.

Because the components of the ActiveMQ architecture are decoupled, you need to monitor producers, consumers, destinations, addresses, and brokers holistically to understand the context of any issues that may arise. For example, if you notice that a producer's output has paused, it may not indicate a problem—but if it correlates with a destination's rising memory usage, it can reveal a bottleneck in the larger system. Later, we'll look at some specific metrics that contribute to the big picture of ActiveMQ monitoring. But first, we'll examine ActiveMQ's fundamental unit of work—the message.

Messages

Each **message** ActiveMQ sends is based on the JMS specification, and is made up of **headers**, optional **properties**, and a **body**.

Headers

JMS message headers contain metadata about the message. Headers are defined in the JMS specification, and their values are set either when the producer creates the message, or when ActiveMQ sends it.

Headers convey qualities of the message that affect how the broker and clients behave. Let's take a look at two key characteristics that ActiveMQ takes into account when delivering messages: expiration and persistence.

MESSAGE EXPIRATION

Depending on its content and purpose, a message may lose its value after a certain amount of time. When a producer creates a message, it can set an expiration value in the message header. If it does not, the header value remains empty and the message never expires.

DASH NYC, June 9-10 | AI + Observability

DATADOG

ActiveMQ messages are persistent by default, but you can configure persistence on a per-message or per-producer basis. When you send a persistent message, the broker saves the message to disk before attempting delivery. If the broker were to crash at that point, a copy of the message would remain and the process of sending the message could recover when the broker restarted. A non-persistent message, on the other hand, usually exists only in the broker's memory and would be lost in an event that caused the broker to restart.

Sending non-persistent messages is usually faster because it doesn't require the broker to execute expensive write operations. Non-persistent messaging is appropriate for short-lived data that gets replaced at frequent intervals, such as a once-a-minute update of an item's location.

Properties

Properties provide a way of adding optional metadata to a message. ActiveMQ supports some properties that are defined in the JMS spec, and also implements some properties that aren't part of the spec.

Producers can also define properties—arbitrarily and outside the JMS spec—and apply them to each message. Consumers can implement selectors to filter messages based on values present in the message properties. For example, you can configure an ActiveMQ producer to attach a `coin` property to each message, with a value of either `heads` or `tails`, and send them all to the same topic. You can write two consumers—a `heads` consumer and a `tails` consumer—that subscribe to that topic but that only receive messages with their selected value of the `coin` property.

Body

The content of an ActiveMQ message is the **body**. The body of a message can be text or binary data. (It's also acceptable for a message's body to be empty.) The value of the `JMSType` message header, which is set explicitly by the producer when the message is created, determines what can be carried in the body of the message: a file, a byte stream, a Java object, a stream of Java primitives, a set of name-value pairs, or a string of text.

For more information about message types, see this JMS documentation.

Memory and storage

ActiveMQ uses memory to store messages awaiting dispatch to consumers. Each message occupies some memory (how much depends on the size of the message) until it is dequeued and delivered to a consumer. At that point, ActiveMQ frees up the memory that had been used for that message. When producers are faster than consumers—there's more enqueuing than dequeuing over a given time period—ActiveMQ's memory use increases.

ActiveMQ also writes messages to disk for storage. Classic and Artemis both use paging to move messages to disk when memory is exhausted. When ActiveMQ needs to send those messages, they're paged from disk back into memory. Paging messages to and from disk adds latency, but it allows ActiveMQ to process a large volume of messages without requiring enough memory to hold them all. Paging is enabled by default, but is optional—you can configure an address to discard messages when there is no memory available to store them.

In this section, we'll look at how ActiveMQ uses memory and disk to store messages.

DASH NYC, June 9-10 | AI + Observability

DATADOG

element in the broker configuration file (**activemq.xml**). You can express this as a percentage of the JVM's heap memory (e.g., `<memoryUsage percentOfJvmHeap="60" />`), or as a number of bytes, as shown below. (Note that your `broker` element may look different than the one in this example, depending on your configuration.)

ACTIVEMQ.XML

```
<broker xmlns="http://activemq.apache.org/schema/core" brokerName="MY_BROKER">
[...]
    <systemUsage>
        <systemUsage>
            <memoryUsage>
                <memoryUsage limit="1 gb" />
            </memoryUsage>
        </systemUsage>
    </systemUsage>
[...]
</broker>
```

The memory specified on the `memoryUsage` element must be shared amongst all of the broker's queues and topics. Each destination may also be configured with an explicit memory limit, designated in the `memoryLimit` element inside an optional `policyEntry` in the **activemq.xml** file:

ACTIVEMQ.XML

```
<broker xmlns="http://activemq.apache.org/schema/core" brokerName="MY_BROKER">
[...]
    <destinationPolicy>
        <policyMap>
            <policyEntries>
                <policyEntry queue="MY_QUEUE" memoryLimit="100mb" />
                <policyEntry topic="MY_TOPIC" memoryLimit="50mb" />
            </policyEntries>
        </policyMap>
    </destinationPolicy>
[...]
</broker>
```

ActiveMQ Artemis uses half of the memory available to the JVM unless you adjust this memory allotment by setting the `global-max-size` parameter in the broker configuration file (**broker.xml**). If the messages held in all of the broker's addresses require all of the space specified by `global-max-size`, Artemis will page new

DASH NYC, June 9-10 | AI + Observability

DATADOG

the defined `global-max-size`.

The code snippet below shows the `address-settings` element from an example file in the ActiveMQ Artemis source code. This configures the paging behavior of two addresses (named "pagingQueue" and "exampleQueue") and includes a catch-all configuration that will apply to all other addresses (designated by `match="#"`). Each address's `max-size-bytes` value determines the maximum amount of memory that will be used to store messages for the address. Artemis uses the `page-size-bytes` value to determine the size of each page file on disk that will store an address's paged messages. As more messages are paged, Artemis will create additional page files of the same size.

BROKER.XML

```
<address-settings>
   <address-setting match="pagingQueue">
      <max-size-bytes>100000</max-size-bytes>
      <page-size-bytes>20000</page-size-bytes>
   </address-setting>

   <address-setting match="exampleQueue">
      <max-size-bytes>10Mb</max-size-bytes>
      <page-size-bytes>1Mb</page-size-bytes>
   </address-setting>

   <address-setting match="#">
      <max-size-bytes>10Mb</max-size-bytes>
      <page-size-bytes>1Mb</page-size-bytes>
   </address-setting>
</address-settings>
```

Both ActiveMQ Classic and Artemis use memory differently for non-persistent messages than they do for persistent messages. Each non-persistent message is stored in memory as it arrives. When the available memory is full, Classic moves all messages to disk, but Artemis leaves existing messages in memory and pages new messages to disk. Each persistent message is also stored in memory as it arrives, and is also written to the message store on disk. If no more memory is available, incoming persistent messages are written directly into the message store.

As long as the memory available to the destination or address isn't exhausted, incoming messages can be dispatched directly from memory without incurring any latency related to disk activity. If the message is not available in memory (either because it got flushed from memory to the temp store or because it was written to the message store when available memory was full), the broker must page the message data from disk in order to dispatch it to a consumer.

Storage

DASH NYC, June 9-10 | AI + Observability

DATADOG

ACTIVEMQ.XML

```
<systemUsage>
    <systemUsage>
        <storeUsage>
            <storeUsage limit="100 mb"/>
        </storeUsage>
    </systemUsage>
</systemUsage>
```

Storage for ActiveMQ Classic's non-persistent messages is specified separately. Non-persistent messages are written to storage only after available memory is exhausted. You can specify the amount of storage to be used for non-persistent messages in the **activemq.xml** file's `tempUsage` element, which defaults to 50 GB. You can configure this as a percentage of available disk space (`percentLimit`) or as a number of bytes (as shown below):

ACTIVEMQ.XML

```
<systemUsage>
    <systemUsage>
        <tempUsage>
            <tempUsage limit="100 mb"/>
        </tempUsage>
    </systemUsage>
</systemUsage>
```

ActiveMQ Classic uses KahaDB as its default message storage mechanism. It stores both persistent and non-persistent messages.

Artemis's primary message store is the file journal. To allow ActiveMQ to manage messages efficiently, the file journal is designed to minimize the movement required of the disk head when HDD storage is used (although ActiveMQ supports SSD storage as well). You can configure the file journal via the **broker.xml** file, as described in the Artemis documentation.

Both versions also support storing messages via JDBC. Using this configuration, you can choose from a number of SQL databases to find the storage mechanism that best meets your needs for scalability and support. See the documentation for ActiveMQ Classic and ActiveMQ Artemis for more information about using the JDBC message store.

We've looked at some characteristics of JMS messages, and at some different ways ActiveMQ stores and sends them. But ActiveMQ's work isn't done until a message is delivered to a consumer. In the next section we'll look at how consumers handle messages.

DASH NYC, June 9-10 | AI + Observability

DATADOG

Durable vs. nondurable subscriptions

A consumer can subscribe to a topic as either a durable or nondurable subscriber. (Durability applies only to messages within a topic, not within a queue.) In the case of a durable subscription, ActiveMQ will retain messages if the subscriber is unavailable. When that subscriber reconnects, it receives new messages that arrived during the time it was disconnected. A nondurable subscriber would not receive any messages published to the topic during the time it was disconnected from the broker.

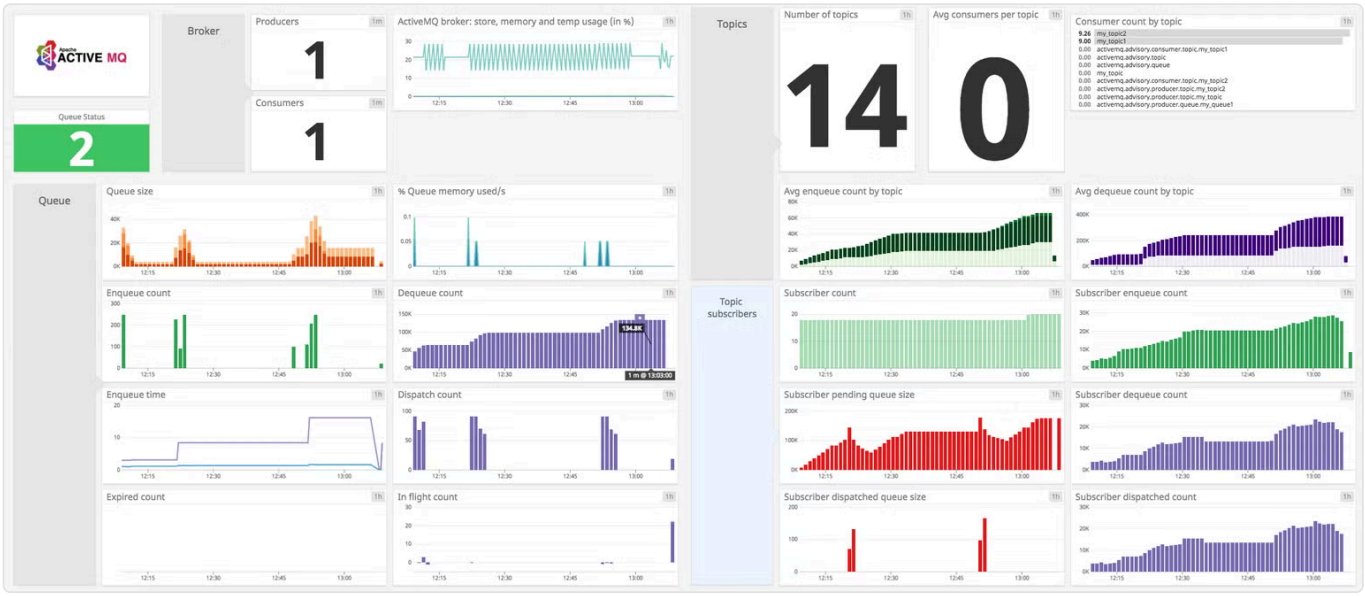
Message acknowledgment

To ensure that messages are received and—if necessary—to avoid sending messages more than once, ActiveMQ supports message acknowledgment. Each consumer is configured to use an acknowledgment mode that determines when and how it will acknowledge a message—either automatically upon receipt, or by making an explicit call to an `acknowledge` method. ActiveMQ metrics for both Classic and Artemis show information about the number of messages acknowledged and not yet acknowledged, but the meaning of those metrics depends on the consumer's acknowledgment mode. A spike in unacknowledged messages could mean that the consumer is offline and unable to receive messages, or that the consumer is failing to successfully execute its manual `acknowledge` call.

So far, we've covered what ActiveMQ is, and how it works. In the next section, we'll introduce some useful metrics to help you understand how to monitor ActiveMQ.

Key metrics for ActiveMQ monitoring

By tracking ActiveMQ metrics, you can effectively monitor resource usage, broker performance, and message activity. Monitoring these metrics can help you understand the performance of your messaging infrastructure and detect potential problems that might affect your services.



ActiveMQ metrics come from:

- destinations or addresses

DASH NYC, June 9-10 | AI + Observability

DATADOG

Because ActiveMQ is written in Java, you can query destination, address, broker, and JVM metrics via Java Management Extensions (JMX). You can view these metrics using JConsole, a GUI that's included in the JDK, or with other JMX-compliant monitoring systems. In Part 2 of this series, we'll look at JConsole and other tools available to help you collect and view ActiveMQ metrics.

In this section, we'll explore key ActiveMQ metrics—where to find them, and the reasons you might want to collect them. This builds on our Monitoring 101 series, which discusses how to identify high-value monitoring data, how to create a smart strategy for alerting, and how to investigate the issues your monitoring uncovers.

Destination and address metrics

ActiveMQ Classic and ActiveMQ Artemis use different types of endpoints to route messages to consumers. Monitoring these endpoints—destinations and addresses—can give you information about the speed, volume, and resource usage of your messaging system.

VERSION	JMX ATTRIBUTE	DESCRIPTION	MBEAN
Classic	`MemoryPercentUsage`	Percentage of configured memory used by the destination	org.apache.activemq:type=Broker,brokerName=\,destinati
Artemis	`AddressSize`	Memory used (in bytes) by the address	org.apache.activemq.artemis:broker="\",component=addr
Classic	`ConsumerCount`	The number of consumers currently subscribed to the destination	org.apache.activemq:type=Broker,brokerName=\,destinati
Artemis	`ConsumerCount`	The number of consumers consuming messages from the queue	org.apache.activemq.artemis:broker="\",component=addr type="anycast",queue="\"
Classic	`ProducerCount`	The number of producers currently	org.apache.activemq:type=Broker,brokerName=\,destinati

DASH NYC, June 9-10 | AI + Observability

DATADOG

		destination	
Classic	`QueueSize`	The number of messages (per destination) that have not been acknowledged by a consumer. Includes those not yet dispatched	org.apache.activemq:type=Broker,brokerName=\,destinati
Artemis	`MessageCount`	The number of messages currently in the queue. Includes scheduled, paged, and in-delivery messages	org.apache.activemq.artemis:broker="\",component=addr type="anycast",queue="\
Classic	`ExpiredCount`	The number of messages in the destination that expired before they could be delivered	org.apache.activemq:type=Broker,brokerName=\,destinati
Artemis	`MessagesExpired`	The number of messages in the queue that expired before they could be delivered	org.apache.activemq.artemis.addresses.\.addresses.queue type="anycast",queue="\

DASH NYC, June 9-10 | AI + Observability

DATADOG

`memoryUsage`, and the `MemoryPercentUsage` metric represents the percentage of that value currently being used by the destination. (See the Memory and storage section for more details.)

ActiveMQ Artemis's `AddressSize` metric measures the memory used by the address.

As the memory usage of a destination or address rises, you may see a decrease in the rate at which your producers send messages. This is thanks to Producer Flow Control (PFC), which reduces the rate at which brokers send messages in both versions of ActiveMQ.

PFC is triggered in ActiveMQ Classic when a destination's memory usage is at or above the `cursorMemoryHighWaterMark` (which defaults to 70 percent of the available memory—either the broker's `memoryUsage` limit or, if defined, the destination's `memoryLimit`). You can change this value by adding an attribute to the relevant `policyEntry` element in **activemq.xml**. The code snippet below shows how you would set `cursorMemoryHighWaterMark` to 80 percent of the broker's memory for a queue named `MY_QUEUE` and 50 percent for a topic named `MY_TOPIC`.

ACTIVEMQ.XML

```
<broker xmlns="http://activemq.apache.org/schema/core" brokerName="MY_BROKER">
[...]
    <destinationPolicy>
        <policyMap>
            <policyEntries>
                <policyEntry queue="MY_QUEUE" cursorMemoryHighWaterMark="80" />
                <policyEntry topic="MY_TOPIC" cursorMemoryHighWaterMark="50" />
            </policyEntries>
        </policyMap>
    </destinationPolicy>
[...]
</broker>
```

See the ActiveMQ documentation for more information about configuring memory limits in ActiveMQ Classic.

If you're using ActiveMQ Artemis, PFC is triggered when an address's memory usage reaches its configured `max-size-bytes` value. The sample code below adds an `address-full-policy` of `BLOCK` to the configuration we created earlier in this post. This policy prevents producers from sending messages to this address when memory is exhausted:

BROKER.XML

```
      <address-settings>
         <address-setting match="exampleQueue">
            <max-size-bytes>10Mb</max-size-bytes>
```

DASH NYC, June 9-10 | AI + Observability

DATADOG

Whichever version of ActiveMQ you're using, PFC could have a noticeable effect on the performance of your messaging system. If you're using ActiveMQ Classic, you should create an alert to notify you when a destination's `MemoryPercentUsage` value approaches its `cursorMemoryHighWaterMark` value so you can take action before PFC is activated. With ActiveMQ Artemis, you should create an alert that triggers when the value of an address's `AddressSize` metric approaches its `max-size-bytes` limit.

If you are using point-to-point messaging, you can reduce memory pressure by scaling out your consumer fleet to dequeue messages more quickly. If your pub/sub messaging system is using durable subscribers, make sure those consumers are available often enough to prevent a backlog of messages. In either case, increasing memory available to your destinations will help, too.

Metric to watch: ConsumerCount

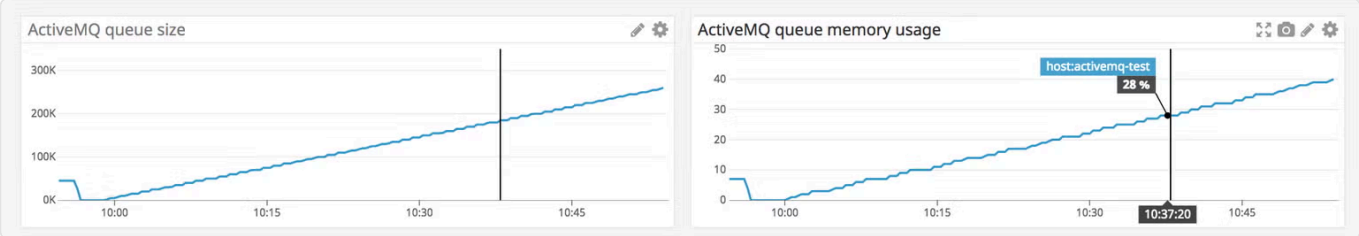
Sooner or later, each destination or address needs to deliver messages to consumers. Consumers may come and go (for example, as your infrastructure dynamically scales), and a fluctuating consumer count could be normal in some cases. You should be able to identify a normal range in the number of connected consumers, and both ActiveMQ versions produce a `ConsumerCount` metric you can monitor for abnormalities. If your `ConsumerCount` value changes unexpectedly, your consumer fleet may have scaled out more than usual, or some hosts may have become unavailable.

Metric to watch: ProducerCount

This metric tracks the number of producers currently attached to an ActiveMQ Classic broker. Whether a `ProducerCount` of zero indicates a problem depends on your expected pattern of activity. If your producers are typically active only sporadically (e.g., if they send a batch of messages once a day), this may be normal. However, if you expect to have active producers at all times, you should investigate a `ProducerCount` of zero, as it could indicate a service interruption.

Metrics to watch: QueueSize (Classic) / MessageCount (Artemis)

These metrics track the number of messages that have not been acknowledged by consumers. If you see this metric consistently increasing, it could indicate that the producers are publishing messages faster than consumers are processing them, or that consumers are failing to acknowledge the messages they receive. This could cause the destination or address to run out of memory (which could even affect the performance of the broker's other destinations or addresses), so you should monitor memory usage metrics alongside these.



DASH NYC, June 9-10 | AI + Observability

DATADOG

decreasing the expiration time of new messages or by removing durable subscribers that are consistently unavailable.

Metrics to watch: ExpiredCount (Classic) / MessagesExpired (Artemis)

These metrics represent the number of messages that expired before they could be delivered. If you expect all messages to be delivered and acknowledged within a certain amount of time, you can set an expiration for each message, and investigate if the number of expiring messages rises above zero.

In some cases, though, expired messages may not be a sign of trouble. For example, if your environment includes consumers with durable subscriptions but an unreliable network, some messages could expire while those consumers are disconnected. When the consumers reconnect, they'll request all messages published in the interim, but if some of those messages contain information that frequently gets updated (e.g., status updates at one-minute intervals), it's better to discard them than deliver them.

Broker metrics

As mentioned earlier, the ActiveMQ broker has configurable limits on the amount of memory and disk space it's allowed to use. Here are some of the metrics you should monitor to ensure that your broker is working properly within its resource limits.

VERSION	JMX ATTRIBUTE	DESCRIPTION	MBEAN
Classic	`MemoryPercentUsage`	Percentage of available memory used by all destinations on the broker	org.apache.activemq:type=Broker,brokerName=\
Artemis	`AddressMemoryUsagePercentage`	Percentage of the broker's available memory (`global-max-size`) used by all the addresses on the broker	org.apache.activemq.artemis:broker=\
Classic	`StorePercentUsage`	Percentage of available disk space (`storeUsage`) used for	org.apache.activemq:type=Broker,brokerName=\

DASH NYC, June 9-10 | AI + Observability

DATADOG

		storage	
Artemis	`DiskStoreUsage`	Percentage of total disk store used	org.apache.activemq.artemis:broker=\
Classic	`TempPercentUsage`	Percentage of available disk space (`tempUsage`) used for non-persistent message storage	org.apache.activemq:type=Broker,brokerName=\
Classic	`TotalEnqueueCount`	The total number of messages sent to the broker	org.apache.activemq:type=Broker,brokerName=\
Artemis	`TotalMessagesAdded`	The total number of messages sent to the broker	org.apache.activemq.artemis:broker=\
Classic	`TotalDequeueCount`	The total number of messages the broker has delivered to consumers	org.apache.activemq:type=Broker,brokerName=\

DASH NYC, June 9-10 | AI + Observability

		messages consumers have acknowledged from all queues on this broker	
Classic	`Topics`	A count of topics currently attached to this broker	org.apache.activemq:type=Broker,brokerName=\
Classic	`Queues`	A count of queues currently attached to this broker	org.apache.activemq:type=Broker,brokerName=\
Artemis	`QueueCount`	The number of queues created on this server	org.apache.activemq.artemis:broker="\
Artemis	`ConnectionCount`	The number of clients connected to the broker	org.apache.activemq.artemis:broker="\"

Metrics to alert on: MemoryPercentUsage (Classic) / AddressMemoryUsagePercentage (Artemis)

If you find that a broker's memory usage is rising, there are a few steps you can take to prevent resource constraints from affecting your application's performance.

- Scale out your consumers. This can increase the rate at which messages are consumed, allowing the broker to reclaim memory and disk space.
- Increase the memory available to the broker. (See the Memory and storage section of this post for information.) To do this, you may need to scale up the amount of memory allocated to the JVM, which could require adding memory to the host.
- Reduce the memory available to the destinations associated with the broker (particularly if you have a large number of destinations). Although this will force the broker to persist messages to disk sooner, it also

DASH NYC, June 9-10 | AI + Observability

DATADOG

usage may help you troubleshoot resource constraints by showing you specifically *now* your host's memory is being consumed.

Metrics to alert on: StorePercentUsage (Classic) / DiskStoreUsage (Artemis)

This is the percentage of available disk space used by the broker's persistent message store. The broker can reach its persistent storage limit if consumers are slow or unavailable, and if messages are large. It's important to monitor this metric because if a broker runs out of persistent storage, PFC may cause producers to stop sending messages.

Metric to watch: TempPercentUsage

When an ActiveMQ Classic broker runs out of memory to store non-persistent messages, it moves those messages to a temp location on the file system. If that temporary storage space fills up, producers will stop sending messages until storage space is freed up (assuming PFC is enabled).

You might run out of temporary storage space for any number of reasons, including:

- Slow or absent consumers.
- Specifying a `tempUsage` value that is smaller than the broker's `memoryUsage`. In this case, the memory holds more message data than the temp store has room for, and `TempPercentUsage` will exceed 100 percent as soon as memory fills and messages are sent to the temp store.
- Specifying a `tempUsage` value that is smaller than KahaDB's `journalMaxFileLength` (which is 32 MB by default). This could cause the temp store to fill up because the broker will create a 32 MB journal file to hold the message data on disk, regardless of the amount of message data in memory.

If PFC is activated, your messaging throughput will drop, so it's important to monitor your `TempPercentUsage` value. You should alert on a value that gives you enough time to remove messages or add disk space before `TempPercentUsage` reaches 100 percent and triggers PFC.

Metrics to watch: TotalEnqueueCount (Classic) / TotalMessagesAdded (Artemis) and TotalDequeueCount (Classic) / TotalMessagesAcknowledged (Artemis)

`TotalEnqueueCount` and `TotalMessagesAdded` track the number of messages sent to the broker. You can monitor these metrics to understand the volume of messages emitted by your producers. `TotalDequeueCount` and `TotalMessagesAcknowledged` show the number of messages that have been delivered by the broker and acknowledged by your consumers. (All of these metrics are cumulative counts calculated over the entire time the broker has been running, and reset to zero when the broker is restarted.)

You should monitor the rate of messages enqueued alongside the dequeue rate to understand your system's overall message volume and the degree to which consumers are keeping up with producers.

Metrics to watch: Topics and Queues (Classic) / QueueCount (Artemis)

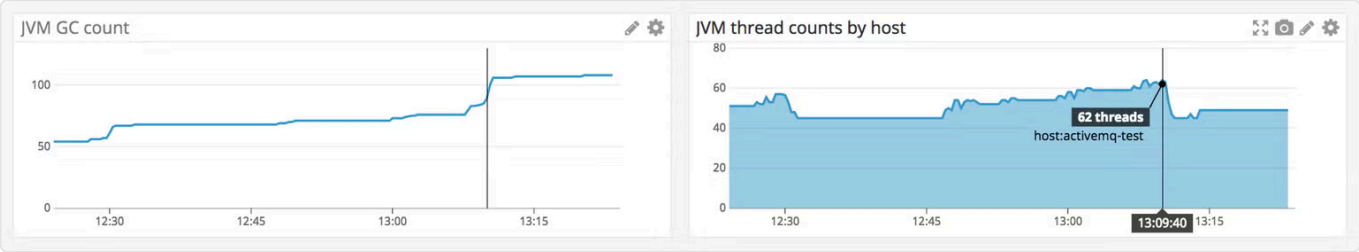
There's no correct number of topics or queues for any given ActiveMQ deployment, but you probably have expectations about what's right for your environment. You can monitor ActiveMQ Classic to see the count of your broker's topics and queues, and you can see the total number of queues on an Artemis broker. These

DASH NYC, June 9-10 | AI + Observability

The number of producers and consumers connected to your Artemis broker might be dynamic, but this metric can be useful for troubleshooting. If `ConnectionCount` falls to zero unexpectedly, it could indicate a networking problem preventing clients from reaching your ActiveMQ server.

JVM metrics

ActiveMQ runs within the JVM, so metrics that inform you of the health of the JVM can be critical in monitoring your messaging. In this section, we'll look at some key JVM metrics.



JMX ATTRIBUTE	DESCRIPTION	MBEAN	METRIC TYPE
`CollectionTime`	The total amount of time (in milliseconds) the JVM has spent executing garbage collection processes	java.lang:type=GarbageCollector,name=(Copy\	MarkSweepCompact
`CollectionCount`	The total count of garbage collection processes executed by the JVM	java.lang:type=GarbageCollector,name=(Copy\	MarkSweepCompact
`HeapMemoryUsage`	This contains values for the heap's `init`, `max`, `committed`, and `used` metrics	java.lang:type=Memory	Resource: Utilization
`ThreadCount`	Threads currently used by the JVM	java.lang:type=Threading	Other

Metric to alert on: CollectionTime

Because ActiveMQ runs in the JVM, its memory is managed by Java's garbage collection (GC) process. A running Java application requires memory to create the objects it uses, and the Java garbage collector periodically evaluates memory usage and frees up unused memory. As ActiveMQ's message volume increases, it will use more memory. As a result, the JVM will execute garbage collection more frequently, which could slow down messaging overall.

DASH NYC, June 9-10 | AI + Observability

DATADOG

collection is happening, and how long each process takes.

In Part 2 of this series, we'll look at some of the tools that use JMX to monitor ActiveMQ. You can use tools like these to watch for an increase in the frequency of GC activity. You can correlate GC activity with any corresponding spikes in the broker's `MemoryPercentUsage` that could explain a slowdown in your messaging activity.

Metric to watch: HeapMemoryUsage

The HeapMemoryUsage metric is a JSON object made up of `init`, `committed`, `max`, and `used` values.

- `init` is set when the JVM starts, and designates the initial amount of heap memory available.
- `max` holds the value of the maximum possible size of the heap.
- `committed` is set by the JVM, and fluctuates. This value indicates how much memory is guaranteed to be available for the JVM to use.
- `used` represents the amount of JVM heap memory currently in use.

You should watch `used` and `committed` together to ensure that the JVM isn't running out of available memory. Java will throw an `OutOfMemoryError` exception if the JVM's memory is exhausted. See the Java documentation and the ActiveMQ FAQ for guidance on resolving this problem.

Metric to watch: ThreadCount

Synchronous messaging requires a greater number of threads than asynchronous delivery. Using more threads causes the broker to incur the overhead of context switching, which requires more work from the host's CPU. This could cause a slowdown in the queueing and dispatching of messages, and ultimately could lead to lower message throughput.

As described in the ActiveMQ documentation, you can reduce the number of threads ActiveMQ Classic requires by using thread pooling, enabling optimized dispatch on your queues, or using the NIO protocol.

Host-level metrics

Your host is the foundation of all the processes involved in ActiveMQ's messaging activities. To understand bottlenecks that may arise, and to make informed decisions about when to scale out, look to your host-level metrics.

NAME	DESCRIPTION	METRIC TYPE
Disk usage	The percentage of the host's available disk space currently in use	Resource: Utilization
Disk I/O	The rate of read and write operations per second	Resource: Utilization

Metric to alert on: Disk usage

ActiveMQ uses disk space to store persistent messages, as well as non-persistent messages that get paged to disk when memory fills up. After a message has been acknowledged by a consumer, ActiveMQ Classic deletes

DASH NYC, June 9-10 | AI + Observability

DATADOG

`TotalMessagesAdded` rate outpaces `TotalMessagesAcknowledged`, your host's disk could fill up. You can correlate host-level disk usage metrics with the `TempPercentUsage` and `StorePercentUsage` metrics on ActiveMQ Classic and the `DiskStoreUsage` metric on Artemis to verify that your disk is filling up with message data instead of, for example, log files from an unrelated process. You can create a forecast alert to notify you that your host is running out of disk space so you can take action in time to prevent performance problems.

Metric to watch: Disk I/O

ActiveMQ Classic brokers store messages on disk in a message store and a temp store. Similarly, Artemis brokers may page message data to disk as memory fills up. As a result, different brokers on the same host may compete for the chance to write messages to the disk. Rising disk I/O could lead to queued write operations, reducing message throughput overall.

If you see high disk activity, it could mean that your broker is very busy, especially if you also see high `MemoryPercentUsage` (from ActiveMQ Classic) or `AddressSize` (from Artemis). If this is the case, you should consider distributing messaging activity across multiple nodes by creating a network of ActiveMQ Classic brokers or a cluster of Artemis servers.

Making meaning of the metrics

ActiveMQ metrics can help you proactively maintain your messaging infrastructure, providing you with information you need to investigate errors, missing messages, and unexpected latency. In this post, we've looked at the metrics you can collect from ActiveMQ, and highlighted some that are particularly valuable to monitor. In Part 2 of this series, we'll show you some of the tools you can use to gather metrics from your ActiveMQ brokers, destinations, and addresses.

Acknowledgments

We'd like to thank Gary Tully of Red Hat for his technical review of this series.

Related jobs at Datadog

We're always looking for talented people to collaborate with

FEATURED POSITIONS

Senior Regional Manager, Commercial Sales Engineering (LATAM)	COMMERCIAL & MID-MARKET SALES ENGINEERING / TECHNICAL SOLUTIONS
Director, Product Management - Infrastructure Monitoring	PRODUCT MANAGEMENT

DASH NYC, June 9-10 | AI + Observability

DATADOG

VIEW ALL

Start monitoring your metrics in minutes

FIND OUT HOW

FREE TRIAL

Download mobile app

Download on the App Store

GET IT ON Google Play

RESOURCES

Pricing
Documentation
Support
Services & Enablement
Product Preview Program
Certification
Open Source
Events and Webinars
Security
Privacy Center
Knowledge Center
Learning Resources

PRODUCT

ABOUT

BLOG

DASH NYC, June 9-10 | AI + Observability



English